

Implementation Plan Duty of Care PNPTv

- IMPLEMENTATION PLAN: DUTY OF CARE ON PNPTV
 - From Statement to Code — Practical, Applicable, and Measurable
 - Executive summary
 - Current state — What already exists (no need to build)
 - PHASE 1 — Privacy sovereignty (creator-level geoblocking)
 - PHASE 2 — Creator-verifiable earnings transparency
 - PHASE 3 — Verifiable identity without custodying PII (migration toward Verifiable Credentials)
 - PHASE 4 — Ethical offboarding (exit protocol)
 - PHASE 5 — Community sovereignty: public transparency panel
 - Suggested timeline
 - Global plan success metrics
 - What's NOT in this plan (and why)
 - Appendix: how to test each phase before going to production

IMPLEMENTATION PLAN: DUTY OF CARE ON PNPTV

From Statement to Code — Practical, Applicable, and Measurable

Version: 1.0

Date: August 2026

Based on: Direct audit of the production repository + reviewed academic thesis

Audience: PNPTv technical team (Santino, Carlos, Miguel)

Executive summary

The technical audit revealed that PNPTv has already implemented the most complex wellness pillars of Duty of Care (Wellness Mode, Use Tracker, § 2257, contextualized content filter). The remaining gaps are around **worker sovereignty**: creators cannot hide their profile from their own country, cannot leave without depending on PNPTv to export their credentials, and cannot verify the earnings ledger without asking an admin for a report.

This plan closes those gaps. It is organized into 3 phases of increasing impact, each with measurable deliverables in code and observable metrics in production.

Current state — What already exists (no need to build)

Area	Module	Status
§ 2257 identity verification	<code>identityVerificationService.js</code>	LIVE
AI age verification	<code>ageVerificationService.js</code>	LIVE
Passkeys / WebAuthn (passwordless)	<code>routes.js</code> + <code>webAppController.js</code>	LIVE
Wellness Mode with 24h cooling-off	<code>wellnessModeService.js</code>	LIVE
Use Tracker (slam/smoke)	<code>routes.js</code> table <code>use_tracker_logs</code>	LIVE
Self-Care Center (no ads, no algo)	<code>SelfCareCenter.tsx</code>	LIVE
PNP-contextualized content filter	<code>contentModerationFilter.js</code>	LIVE
Report system (8 categories + auto CSAM)	<code>userReportService.js</code>	LIVE
Multi-vector ban	<code>platformBanService.js</code>	LIVE
Multi-crypto cashout (BTC/DASH/USDT/Meru)	<code>cashoutService.js</code>	LIVE
Ru\$h append-only ledger	<code>tokenLedgerService.js</code>	LIVE
		LIVE

Area	Module	Status
Self-service right to erasure	<code>selfEraseAccount</code> in <code>usersController</code>	
HMAC location fuzzing	<code>nearbyService.js</code>	LIVE
Public appeal system	<code>appealService.js</code>	LIVE

PHASE 1 — Privacy sovereignty (creator-level geoblocking)

Priority: CRITICAL

Estimated effort: 3-5 days

Owner: _____

Due date: _____

Impact: Protects creators from family ostracism, persecution in hostile countries, loss of conventional employment

Why now

Country of origin is the most common risk vector for creators in Latin America. A creator in Colombia, Mexico, or Venezuela who has family or a conventional job cannot control whether a family member who searches their name on PNPTv lands on their profile. This feature is the most requested on similar platforms and the most absent.

1.1 — Geoblocking configuration table

```
-- Migration: add creator geo-privacy settings
ALTER TABLE users ADD COLUMN IF NOT EXISTS geo_block_countries TEXT[] DEFAULT '{}';
-- Example: ['CO', 'MX', 'VE'] – ISO 3166-1 alpha-2 codes
```

```
-- Empty array = visible globally (default, no change in behavior)

CREATE INDEX IF NOT EXISTS idx_users_geo_block ON users USING
    GIN(geo_block_countries)
WHERE array_length(geo_block_countries, 1) > 0;
```

1.2 — Geoblocking middleware on public profiles

File to modify: `apps/backend/bot/api/middleware/ipTracker.js` (or create a new `geoBlockMiddleware.js`)

```
// apps/backend/services/geoBlockService.js
'use strict';

const { query } = require('../config/postgres');
const logger = require('../utils/logger');

/**
 * Resolves country code from request using existing geo detection.
 * Falls back to null if unresolvable.
 */
function resolveCountry(req) {
    // geo detection already exists in subscriptionController – centralize here
    const forwarded = req.headers['x-forwarded-for'];
    const ip = forwarded ? forwarded.split(',')[0].trim() : req.ip;
    // Use maxmind/ipinfo if configured, else fall back to req.geo set by
    // upstream middleware
    return req.geo?.country || req.session?.user?.country || null;
}

/**
 * Check if a creator's profile is blocked for a given country.
 * Returns true if the requesting country is in the creator's block list.
 */
async function isBlockedForCountry(creatorId, requestingCountry) {
    if (!requestingCountry) return false; // Can't determine country – allow

    const { rows } = await query(
        `SELECT geo_block_countries FROM users WHERE id = $1 LIMIT 1`,
        [creatorId]
    );

    const blockList = rows[0]?.geo_block_countries || [];
    return blockList.includes(requestingCountry.toUpperCase());
}
```

```

/**
 * Express middleware factory for geo-blocking a creator's public profile.
 * Usage: router.get('/profile/:creatorId', geoBlockMiddleware('creatorId'),
 *           handler)
 */
function geoBlockMiddleware(creatorIdParam = 'creatorId') {
  return async (req, res, next) => {
    const creatorId = req.params[creatorIdParam] || req.query[creatorIdParam];
    if (!creatorId) return next();

    const country = resolveCountry(req);

    try {
      const blocked = await isBlockedForCountry(creatorId, country);
      if (blocked) {
        // Return the same 404 as a non-existent profile – never confirm the
        // profile exists
        return res.status(404).json({ error: 'Profile not found' });
      }
      next();
    } catch (err) {
      logger.error('geoBlockMiddleware error (fail-open)', { creatorId,
        error: err.message });
      next(); // Fail open – don't lock users out on a DB error
    }
  };
}

module.exports = { resolveCountry, isBlockedForCountry, geoBlockMiddleware };

```

1.3 — API for creators to configure their list

File to modify: `apps/backend/bot/api/routes.js` — add in the creator routes section

```

// In routes.js, in the creator settings section
app.get('/api/webapp/creator/geo-privacy', requireSessionAuth,
  asyncHandler(async (req, res) => {
    const { rows } = await pool.query(
      'SELECT geo_block_countries FROM users WHERE id = $1',
      [req.session.user.id]
    );
    return res.json({ blockedCountries: rows[0]?.geo_block_countries || [] });
  }));

app.put('/api/webapp/creator/geo-privacy', requireSessionAuth,
  asyncHandler(async (req, res) => {

```

```

const { countries } = req.body;
if (!Array.isArray(countries)) return res.status(400).json({ error:
  'countries must be an array of ISO codes' });

// Validate: ISO 3166-1 alpha-2 only, max 50 countries
const VALID_ISO = /^[A-Z]{2}$/;
const validated = countries
  .map(c => String(c).toUpperCase().trim())
  .filter(c => VALID_ISO.test(c))
  .slice(0, 50);

await pool.query(
  'UPDATE users SET geo_block_countries = $1 WHERE id = $2',
  [validated, req.session.user.id]
);
return res.json({ success: true, blockedCountries: validated });
}));

```

1.4 — UI in Creator Settings

File to modify: `apps/web/src/pages/creators/CreatorSettings.tsx`

Add a “Geographic privacy” section with: - Country selector with flags (full ISO list, multiselect) - Quick toggle “Hide my profile in my home country” (auto-detected by IP at time of configuration) - Warning: “Your profile will show ‘not found’ for visitors from the selected countries”

1.5 — Apply the middleware on public profile routes

Files to modify: Routes that expose a creator’s public profile (look for `GET /api/webapp/profile/:username` or similar).

Success metrics

- **Technical:** 0 5xx errors when activating geo-block; profile returns 404 from a blocked country’s IP
- **Adoption:** $\geq$ 15% of active creators configure at least 1 blocked country within the first 30 days

- **Privacy:** 0 reports of “my family found my profile” post-implementation (vs. current baseline)

PHASE 2 — Creator-verifiable earnings transparency

Priority: HIGH

Estimated effort: 2-3 days

Owner: _____

Due date: _____

Impact: Creators can audit their own split without asking anything of an admin

Why now

The Ru\$h ledger (`token_ledger`) exists and is append-only. The problem: creators have no UI view of it. Currently `CreatorEarnings.tsx` shows totals but not the ledger transaction by transaction. This creates a trust dependency (“I trust the 70% is really 70%”) instead of direct verification.

2.1 — Creator’s own ledger endpoint

File to modify: `apps/backend/bot/api/routes.js`

```
// Creator earnings ledger – full transaction history, paginated
app.get('/api/webapp/creator/earnings/ledger', requireSessionAuth,
  asyncHandler(async (req, res) => {
    const userId = req.session.user.id;
    const page = Math.max(1, parseInt(req.query.page) || 1);
    const limit = Math.min(100, parseInt(req.query.limit) || 50);
    const offset = (page - 1) * limit;

    const { rows } = await pool.query(
      `SELECT
        tl.id,
```



```

        tl.created_at,
        tl.reason,
        tl.balance_delta,
        tl.gifted_delta,
        tl.balance_after,
        tl.source_type,
        tl.source_id,
        tl.metadata
    FROM token_ledger tl
    WHERE tl.user_id = $1
    ORDER BY tl.created_at DESC
    LIMIT $2 OFFSET $3`,
    [userId, limit, offset]
);

const { rows: countRows } = await pool.query(
    'SELECT COUNT(*) AS total FROM token_ledger WHERE user_id = $1',
    [userId]
);

return res.json({
    ledger: rows,
    total: parseInt(countRows[0].total),
    page,
    limit,
    commissionRate: 0.30,
    creatorRate: 0.70,
});
}));

```

2.2 — Ledger view in CreatorEarnings

File to modify: `apps/web/src/pages/creators/CreatorEarnings.tsx`

Add a “Full history” tab with a paginated table: - Columns: Date, Description, Tokens received, Balance after - Export to CSV (button) - Filters: by type (tip, content, subscription, cashout) - Explanatory tooltip for the 70/30 split on each `membership_purchase` / `content_purchase` row

2.3 — Earnings breakdown by category

```
// Summary endpoint – quick stats for the earnings dashboard
app.get('/api/webapp/creator/earnings/summary', requireSessionAuth,
  asyncHandler(async (req, res) => {
    const userId = req.session.user.id;

    const { rows } = await pool.query(
      `SELECT
        reason,
        COUNT(*) AS count,
        SUM(balance_delta) AS total_tokens,
        MIN(created_at) AS first_at,
        MAX(created_at) AS last_at
      FROM token_ledger
      WHERE user_id = $1 AND balance_delta > 0
      GROUP BY reason
      ORDER BY total_tokens DESC`,
      [userId]
    );

    return res.json({ breakdown: rows, tokenRateUsd: 1/6 });
  }));
```

Success metrics

- **Technical:** Endpoint responds < 200ms for ledgers with up to 10,000 rows
 - **Adoption:** >= 40% of active creators visit the ledger at least once per month
 - **Trust:** >= 50% reduction in “I wasn’t paid correctly” support tickets (measurable vs. Slack #ext-* baseline)
-

PHASE 3 — Verifiable identity without custodying PII (migration toward Verifiable Credentials)

Priority: MEDIUM-HIGH

Estimated effort: 2-4 weeks (includes external integration)

Owner: _____

Due date: _____

Impact: Eliminates the largest PII concentration point on the platform

Why now

Identity documents stored on PNPTv's server (`id_document_path` , `id_selfie_path`) are the platform's biggest privacy risk. A breach at this point would expose government documents of adult performers — the worst possible institutionalized doxxing scenario.

3.1 — Full migration to Persona (eliminate direct upload)

Persona.com is already integrated in `identityVerificationService.js` (`startPersonaInquiry` , `handlePersonaWebhook`). The alternative flow (manual document upload) must be deprecated.

Steps: 1. Disable direct document upload routes in production 2. Redirect 100% of the verification flow to `startPersonaInquiry` 3. The `id_document_path` and `id_selfie_path` fields in `creator_2257_records` remain as references to `persona_inquiry_id` (already implemented), not as local file paths 4. Purge existing files on disk with an audited migration script

Migration script:

```
// scripts/migrate-2257-to-persona-refs.js
// Replaces local file paths with Persona inquiry references
// Only for already-approved records – documents are unnecessary once approved
const { query } = require('../config/postgres');
```

```

async function migrate() {
  const { rows } = await query(
    `SELECT id, user_id, id_document_path, id_selfie_path, persona_inquiry_id
    FROM creator_2257_records
    WHERE verification_status = 'approved'
    AND persona_inquiry_id IS NOT NULL`
  );

  for (const record of rows) {
    // Null out local paths – Persona is the authoritative record
    await query(
      `UPDATE creator_2257_records
      SET id_document_path = NULL, id_selfie_path = NULL
      WHERE id = $1`,
      [record.id]
    );
    // Log for audit trail
    console.log(`Migrated record ${record.id} for user ${record.user_id}`);
  }
  console.log(`Migrated ${rows.length} records.`);
}
migrate();

```

Success metrics: - 0 new document files on disk post-migration -
 Persona integration rate $\geq$ 95% of new verifications - Median
 verification time $\leq$ 48h (vs. current which depends on manual review)

3.2 — Portable data export for creators (data portability)

Creators must be able to export all their data in a portable format before closing their account.

File to modify: `apps/backend/bot/api/routes.js`

```

// Data export – GDPR-style, creator-initiated
app.post('/api/webapp/account/export', requireSessionAuth, asyncHandler(async
  (req, res) => {
    const userId = req.session.user.id;

    // Rate limit: 1 export per 24h
    const lastExport = await redis.get(`data_export:${userId}`);
    if (lastExport) return res.status(429).json({ error: 'Export available once
    per 24h' });
  }
);

```

```

// Gather all creator data
const [profile, ledger, content, earnings, subs] = await Promise.all([
  pool.query('SELECT username, first_name, last_name, email, bio,
    created_at FROM users WHERE id = $1', [userId]),
  pool.query('SELECT * FROM token_ledger WHERE user_id = $1 ORDER BY
    created_at', [userId]),
  pool.query('SELECT id, title, media_type, created_at, is_premium FROM
    creator_media WHERE creator_id = $1', [userId]),
  pool.query('SELECT * FROM creator_earnings WHERE creator_id = $1 ORDER BY
    created_at', [userId]),
  pool.query('SELECT * FROM creator_subscriptions WHERE creator_id = $1',
    [userId]),
]);

const exportData = {
  exported_at: new Date().toISOString(),
  profile: profile.rows[0],
  ledger: ledger.rows,
  content: content.rows,
  earnings: earnings.rows,
  subscriptions: subs.rows,
};

await redis.setex(`data_export:${userId}`, 86400, '1');

res.setHeader('Content-Disposition', `attachment; filename="pnptv-data-${
  {userId}.json"`);
res.setHeader('Content-Type', 'application/json');
return res.json(exportData);
}));

```

UI: “Export my data” button in `CreatorSettings.tsx` and on the account deletion screen.

Metrics: - Feature available without support dependency - Generation time < 5 seconds for 95% of creators

PHASE 4 — Ethical offboarding (exit protocol)

Priority: MEDIUM

Estimated effort: 1-2 weeks

Owner: _____

Due date: _____

Impact: Covers the only classic HR gap the code doesn't address: the exit

Why it matters

The current code has `selfEraseAccount` (hard delete, irreversible). But it has no exit protocol that includes: delivery of pending earnings, notification to active subscribers, grace period to download their own content. A creator who deletes their account today loses held earnings if any exist.

4.1 — Account deletion pre-flight

File to modify: `apps/backend/bot/api/controllers/usersController.js`

— modify `selfEraseAccount`

```
// Add pre-flight check before deleting
app.get('/api/webapp/account/erase-preflight', requireSessionAuth,
  asyncHandler(async (req, res) => {
    const userId = req.session.user.id;

    const [earnings, subs, hold] = await Promise.all([
      pool.query(
        'SELECT COALESCE(SUM(amount_creator),0) AS available FROM creator_earnings WHERE creator_id=$1 AND status=\''available\'',
        [userId]
      ),
      pool.query(
        'SELECT COUNT(*) AS active FROM creator_subscriptions WHERE creator_id=$1 AND expires_at > NOW()',
        [userId]
      ),
      pool.query(
        'SELECT COALESCE(SUM(amount_creator),0) AS held FROM creator_earnings WHERE creator_id=$1 AND status=\''held\'',
        [userId]
      ),
    ]);

    return res.json({
      availableEarnings: parseFloat(earnings.rows[0].available),
```

```

heldEarnings: parseFloat(hold.rows[0].held),
activeSubscribers: parseInt(subs.rows[0].active),
warning: parseInt(subs.rows[0].active) > 0
  ? `Tienes $
    {subs.rows[0].active} suscriptores activos. Serán notificados y
    reembolsados automáticamente.`
  : null,
earningsWarning: parseFloat(earnings.rows[0].available) > 0
  ? `Tienes $$${earnings.rows[0].available} disponibles para retiro. Haz
    cashout antes de borrar tu cuenta.`
  : null,
});
}));

```

4.2 — Step-by-step offboarding UI

File to modify: Add flow in `apps/web/src/pages/Settings.tsx` or create `OffboardingFlow.tsx`

Step 1: Pre-flight (show pending earnings, active subscribers)

Step 2: One-click cashout options if there is a balance

Step 3: Confirmation with text field “DELETE MY ACCOUNT”

Step 4: Bilingual confirmation (ES/EN)

4.3 — Subscriber notification on account deletion

File to modify: `apps/backend/bot/api/controllers/usersController.js`
— in `hardDeleteUser`

Add notification to active subscribers before deletion:

```

// In hardDeleteUser, before DELETE FROM users:
const { rows: activeSubs } = await client.query(
  'SELECT subscriber_id FROM creator_subscriptions WHERE creator_id=$1 AND
    expires_at > NOW()',
  [userId]
);

// Notify each subscriber
for (const sub of activeSubs) {
  await sendSystemDM('system', sub.subscriber_id,
    'Un creador que seguías ha cerrado su cuenta en PNptv. Tu suscripción ha
    sido cancelada y el tiempo no utilizado será acreditado.',
    client
  );
}

```

```
);  
}
```

Success metrics: - 0 support tickets from creators who deleted their account without withdrawing earnings - 100% of subscribers notified when a creator leaves - Offboarding time (from start of flow to confirmed deletion): median < 10 minutes

PHASE 5 — Community sovereignty: public transparency panel

Priority: LOW-MEDIUM

Estimated effort: 1 week

Owner: _____

Due date: _____

Impact: External verifiability of the model; relevant for the academic thesis and for regulators

Why it matters

The academic document noted that the promise of a cooperative model requires verifiable transparency. An on-chain ledger is not necessary to take this step; publishing distribution statistics that any observer can read is sufficient.

5.1 — Public transparency endpoint

```
// Public transparency endpoint – no auth required  
app.get('/api/public/transparency', asyncHandler(async (req, res) => {  
  // Cache 1 hour  
  const cached = await redis.get('transparency:stats');  
  if (cached) return res.json(JSON.parse(cached));  
  
  const [split, wellness, moderation] = await Promise.all([
```



```

pool.query(`
  SELECT
    COUNT(DISTINCT creator_id) AS active_creators,
    SUM(amount_creator) AS total_paid_to_creators,
    SUM(amount_platform) AS total_platform_commission,
    ROUND(AVG(amount_creator / NULLIF(amount_creator + amount_platform,
    0)) * 100, 1) AS avg_creator_pct
  FROM creator_earnings
  WHERE status = 'paid_out'
    AND created_at > NOW() - INTERVAL '90 days'
`),
pool.query(`
  SELECT
    COUNT(*) AS wellness_sessions,
    SUM(wellness_days_accumulated) AS total_wellness_days
  FROM users
  WHERE wellness_days_accumulated > 0
`),
pool.query(`
  SELECT category, COUNT(*) AS reports
  FROM user_reports
  WHERE created_at > NOW() - INTERVAL '90 days'
  GROUP BY category
  ORDER BY reports DESC
`),
]);

const data = {
  updated_at: new Date().toISOString(),
  period: 'last_90_days',
  creator_economy: {
    active_creators: parseInt(split.rows[0].active_creators),
    creator_share_pct: parseFloat(split.rows[0].avg_creator_pct) || 70,
    platform_commission_pct: 30,
    total_paid_to_creators_usd:
      parseFloat(split.rows[0].total_paid_to_creators) || 0,
  },
  wellness: {
    total_wellness_sessions: parseInt(wellness.rows[0].wellness_sessions),
    total_wellness_days_accumulated:
      parseInt(wellness.rows[0].total_wellness_days),
  },
  moderation: {
    reports_by_category: moderation.rows,
  },
};

await redis.setex('transparency:stats', 3600, JSON.stringify(data));

```

```
return res.json(data);  
}));
```

5.2 — Public transparency page

File: `apps/web/src/pages/Transparency.tsx` (new — exception to no-new-files: this is a public content page, not a component)

Display the endpoint metrics on a page accessible without login, referenceable from the thesis and from press releases.

Success metrics: - Page indexed by Google - Cited in at least 1 academic or regulatory document - Data updated every hour without manual intervention

Suggested timeline

Week	Phase	Deliverable	Owner	Due date
1	Phase 1	SQL migration + geoblock middleware + API endpoint	—	—
2	Phase 1	Creator Settings UI + QA on staging	—	—
3	Phase 2	Ledger endpoint + CreatorEarnings UI	—	—
4	Phase 3	100% Persona migration +	—	—

Week	Phase	Deliverable	Owner	Due date
		document purge script		
5-6	Phase 3	Data portability export + QA	—	—
7	Phase 4	Offboarding pre-flight + offboarding UI flow	—	—
8	Phase 5	Transparency endpoint + public page	—	—

Global plan success metrics

Metric	How to measure	Target 90 days post-launch
Creators with active geoblocking	<code>SELECT COUNT(*) FROM users WHERE array_length(geo_block_countries,1) > 0</code>	>= 15% of active creators
Creators who visited their ledger	Analytics on CreatorEarnings (new tab)	>= 40% creator MAU
Support tickets for “I wasn’t paid correctly”	Count in Slack #ext-*	>= 50% reduction
	<code>find /opt/pnptvapp/uploads -name "*.jpg" -newer migration-date</code>	0 new documents

Metric	How to measure	Target 90 days post-launch
Local documents on disk post-migration		
Offboardings with prior cashout	<code>SELECT COUNT(*) FROM cashout_requests WHERE created_at BETWEEN erase_started AND erase_completed</code>	>= 80% of creators with balance > \$0
Transparency page active	HTTP 200 on <code>/transparency</code>	Yes, no SLA breach

What's NOT in this plan (and why)

Item	Reason for exclusion
Native SSI/DIDs/ZKP	Maturing technology; integration complexity 3-6 months; Persona already covers 90% of the practical benefit
Stealth panic button	PNPtv is a digital platform, not in-person work coordination; physical in-person risk is not the primary use case
On-chain ledger (blockchain)	Gas cost + latency + terrible UX for the average user; the append-only DB ledger with CSV export is sufficient for practical verifiability
Formal cooperative structure	

Item	Reason for exclusion
	Requires corporate reform, not code; outside the scope of the technical team
Colombian ARL / social security	Requires labor intermediation license; outside technical scope

Appendix: how to test each phase before going to production

Phase 1 — Geoblocking

```
# Simulate request from Colombia (CO)
curl -H "X-Forwarded-For: 181.51.0.1" \
      https://pnptv.app/api/webapp/profile/[username_with_CO_blocked]
# Expected: 404

# Simulate request from another country
curl -H "X-Forwarded-For: 50.1.2.3" \
      https://pnptv.app/api/webapp/profile/[username_with_CO_blocked]
# Expected: 200 with profile
```

Phase 2 — Ledger

```
# Verify that the ledger total matches creator_earnings
docker exec pg-pnptv psql -U pnptvbot -d pnptvbot -c "
SELECT
    tl.total_ledger,
    ce.total_earnings,
    ABS(tl.total_ledger - ce.total_earnings) AS discrepancy
FROM
    (SELECT SUM(balance_delta) AS total_ledger FROM token_ledger WHERE
     user_id='[id]' AND reason LIKE '%receive%') tl,
```

```
(SELECT SUM(amount_creator) AS total_earnings FROM creator_earnings WHERE  
creator_id=[id]) ce;  
"
```

Phase 3 — Documents

```
# Verify no local paths remain in approved records  
docker exec pg-pnptv psql -U pnptvbot -d pnptvbot -c "  
  SELECT COUNT(*) FROM creator_2257_records  
  WHERE verification_status='approved'  
    AND (id_document_path IS NOT NULL OR id_selfie_path IS NOT NULL);  
"  
# Expected: 0
```

This plan was generated based on a direct audit of the production repository in August 2026. Every code block is directly applicable to the current stack (Node.js + Express + PostgreSQL + Redis + React/Vite). No technology or architecture changes are proposed.